

Access Specifiers

The Security Guard of C++

Defining Visibility and Data Protection

PUBLIC • PRIVATE • PROTECTED

1. Why Access Specifiers?

In a professional program, you don't want every part of the code to "meddle" with critical data. Access specifiers implement **Encapsulation**—the idea of hiding complexity and protecting the internal state of an object.

Analogy of a Bank:

- **Public:** The ATM in the lobby. Anyone can walk up and use it.
- **Private:** The internal vault software. Only the bank itself can access it.
- **Protected:** The employee breakroom. Only the bank and its partner branches (children) can enter.

2. Open Communication

Members marked as `public` are accessible from **anywhere** in your program. This is the interface through which users interact with your class.

```
class CodeHive {
public:
    string courseName; // Direct access allowed

    void startCourse() {
        cout << "Course Started!";
    }
};

int main() {
    CodeHive obj;
    obj.courseName = "C++"; // Works!
    obj.startCourse();      // Works!
}
```

3. Maximum Protection

Members marked as `private` are completely hidden. Only functions **inside the same class** can touch them. By default, all class members in C++ are private if you don't specify anything.

```
class CodeHive {
private:
    int password; // Secret!

public:
    void login() {
        password = 1234; // Allowed (Inside class)
    }
};

int main() {
    CodeHive obj;
    // obj.password = 0000; // ❌ ERROR: Private
}
```

4. The Controlled Gateway

Since we can't touch private data directly, we use **Getters** (to read) and **Setters** (to write). This allows us to validate data before it's saved.

```
class CodeHive {  
private:  
    int duration;  
public:  
    void setDuration(int d) {  
        if (d > 0) duration = d; // Validation!  
    }  
    int getDuration() { return duration; }  
};
```

Benefit: If the duration must never be negative, the Setter prevents a bug from ever entering your data.

5. The Inheritance Exception

A protected member is private to the outside world, but it acts like a public member to **child classes** (derived classes).

```
class BaseCourse {
protected:
    string id = "ABC-123";
};

class CodeHive : public BaseCourse {
public:
    void showId() {
        cout << id; //  Allowed: Child can see it
    }
};
```

6. Visibility Matrix

This table acts as a cheat sheet for deciding which specifier to use based on the level of interaction required.

Scenario	Public	Private	Protected
Inside Class	✓	✓	✓
Outside (main)	✓	✗	✗
Child Class	✓	✗	✓

Golden Rule: Always start with `private`. Only make a member `public` if you are 100% sure it needs to be seen by the rest of the world.

7. Professional Standards

1. The Data Hide Rule

Variables should almost always be **private**. Methods that manipulate those variables should be **public**. This separates "what" an object does from "how" it stores data.

2. Helper Methods

If you write a method that is only used by your other methods (like a complex math calculation used inside a print function), make it **private**. Don't clutter the user's interface.

8. Encapsulation Summary

By using access specifiers, you create a "Black Box." Other developers use your code by calling public methods without needing to understand the private mess inside.

```
class Engine {  
private:  
    void igniteSpark() { /* ... */ }  
    void pumpFuel() { /* ... */ }  
  
public:  
    void start() {  
        igniteSpark();  
        pumpFuel();  
        cout << "Engine Vroom!";  
    }  
};  
// User only sees obj.start()!
```

9. Skill Check: The Secure Safe

The Task: Imagine a "Vault" class.

1. Make the `int balance` private.
2. Create a protected `string vaultCode` for family-brand vaults.
3. Create a public method `withdraw()` that checks if balance is enough.

```
// Write your logic here ...  
class Vault { ... };
```

SECURITY VERIFIED: ACCESS SPECIFIERS MASTERED